Assignment 4

CMPT 276: Introduction to Software Engineering
Simon Fraser University

Keith
301625387

March 31, 2026

Code Smells Found Reviewing Brian's Code

1 Repeated `onCollect` Boilerplate with No Idempotency Guard

Location: *`Interactable.java`, `CashStack.java`, `InkDart.java`, `RareDiamond.java`, `GlueTrap.java`*

Every subclass ends `onCollect()` by setting `isCollected = true`, and none of them check the flag before applying effects. So this can happen repeatedly: `CashStack.onCollect` adds balance, decrements static money, then sets `isCollected` true; `InkDart.onCollect` applies penalty, then sets `isCollected` true; `RareDiamond.onCollect` adds balance, restarts diamond timer, then sets `isCollected` true; `GlueTrap.onCollect` applies freeze, then sets `isCollected` true.

Why it is a problem

`isCollected` is only written, never effectively read, so an item can be applied multiple times. This is both duplication and state misuse: each subclass reimplements the same "post collect bookkeeping," and without a guard, repeated calls keep rewarding/penalizing.

Refactoring (Template Method + idempotency guard)

In `Interactable`, define one `final collect` method that handles the guard and state transition, and keep item-specific behavior in a protected hook. The method returns immediately if `isCollected` is already true, otherwise calls `onCollectEffect(board)` and then sets `isCollected = true`. `onCollectEffect` is declared as a `protected abstract` method. Then each subclass implements only `onCollectEffect`.

2 Inconsistent Field Naming and Visibility

Location: *`CashStack.java`, `InkDart.java`, `RareDiamond.java`*

The three value-carrying subclasses use different names and access modifiers for the same concept (a monetary effect):

- **CashStack** - field `value`, modifier `protected`
- **InkDart** - field `penalty`, modifier `private`
- **RareDiamond** - field `value`, modifier `private`

Why it is a problem

This inconsistency makes the code harder to read and maintain. A new developer has no "uniform" way to identify "the money/value effect" across items. Also, `protected` on `CashStack.value` suggests subclassing intent that isn't present in the other classes.

Refactoring

Pull a single `protected int value` field into `Interactable`. Each subclass sets it via `super(value)` or in its own constructor and removes duplicate declarations.

3 Magic Numbers

Location: CashStack.java, InkDart.java, RareDiamond.java, GlueTrap.java

There are hard-coded numeric literals with no named constants: `CashStack`: `this.value = 100`; `InkDart`: `this.penalty = -150`; `RareDiamond`: `this.value = 500`; `GlueTrap`: `board.applyFreeze(1)`.

Why it is a problem

These values don't explain intent at the call site. If balancing changes, you have to hunt through constructors. For a new developer, the `1` in `GlueTrap` is especially unclear - does it mean 1 tick, 1 second, or 1 move?

Refactoring

Replace literals with named constants in a dedicated `GameItemConstants` class: `CASH_STACK_VALUE = 100`, `RARE_DIAMOND_VALUE = 500`, `INK_DART_PENALTY = 150`, and `GLUE_STICK_TICKS = 1`, each annotated with a comment identifying the relevant class.

4 High Coupling - `RareDiamond` Touches Board Timer Logic

Location: RareDiamond.java, Board.java

`RareDiamond.onCollect()` calls `board.restartDiamondTimer()`, reaching into `Board` timer/spawn internals. No other `Interactable` subclass directly touches the board's timer/spawn system. `CashStack` does balance and static money decrement, `InkDart` does balance, `GlueTrap` does freeze - all straightforward effects. `RareDiamond` is the outlier.

Why it is a problem

This is high coupling. A collectible should not decide when the next diamond spawns. If spawn logic changes (e.g. to cooldown mode), `RareDiamond` has to change even though its reward behavior is unchanged. It also makes `RareDiamond` harder to unit test independently.

Refactoring

Move `restartDiamondTimer()` out of `RareDiamond` and into `Board.moveThief()`, where post-collection cleanup already belongs. After `triggeredItem.getInteractable().onCollect(this)`, the board checks whether the triggered item was `activeDiamond`, sets it to `null`, and calls `restartDiamondTimer()` directly - keeping timer ownership within `Board`.

Code Smells Brian Found in My Code and My Solutions

1 Repeated Font Creation

Location: Interface.java

`Interface.java` was calling `new Font("Arial", ...)` inline inside drawing methods. Since these methods run inside `paintComponent`, the same font objects were being recreated repeatedly at runtime instead of once. The font family string `"Arial"` also appeared as a repeated raw literal with no central constant.

Why it is a problem

Object allocation in rendering paths can hurt performance and reduce readability. Repeating the literal `"Arial"` also made it easy to introduce inconsistent font usage if one call were later changed but others were missed.

Refactoring Performed

Identified all `g.setFont(new Font(...))` calls across drawing methods in `Interface.java`. Extracted each distinct font into `private static final Font` constants at the top of the class. Extracted the family name into a single `private static final String FONT_FAMILY` constant. Replaced inline `new Font(...)` calls with the named constants. The six paint tests in `InterfaceTest` - `testPaintMainMenuDoesNotThrow`, `testPaintPlayingDoesNotThrow`, `testPaintPausedDoesNotThrow`, `testPaintWinDoesNotThrow`, `testPaintGameOverDoesNotThrow`, and `testPaintTutorialDoesNotThrow` - along with `testRapidStateChangesDoNotThrow` all passed, confirming rendering behavior was unchanged.
Commit `4939b44`: "reactor: ui extract shared font constants + decouple interface from board with session contracts"

2 Tight Coupling with `Board`

Location: `Interface.java`, `Board.java`

`Interface.java` held a direct reference to `Board` and called board methods directly (`board.setPaused()`, `board.reset()`, `board.getBalance()`). There was no abstraction representing what the UI needed from game logic, and `Interface` couldn't be tested in isolation without creating a full `Board` instance.

Why it is a problem

UI was tightly coupled to concrete game implementation, violating separation of concerns and increasing fragility. Any `Board` API change could break UI code. It also made interface testing harder and reversed dependency flow for game outcome signaling.

Refactoring Performed

Grouped all `board.*` usages in `Interface` by responsibility (state control vs. data access). Introduced `GameSession` interface for UI consumption and `GameEvents` interface for game-to-UI callbacks. Updated `Board` to implement `GameSession` and wire callback registration through `setGameEvents`. Updated `Interface` to depend on `GameSession` instead of concrete `Board`. Kept backward-compatible `Interface(Board board)` constructor. `InterfaceTest.testBoardIsPausedWhenNotPlaying`, `testSetStatePlaying`, `testSetStatePaused`, and `BoardTest.testSetPausedTrue` / `testSetPausedFalse` all passed, confirming the session contract preserved all pause and state behaviour.
Commit `6a3c3d9`: "refactor: extract GameInterfacePanel and connect board through GameSession contracts"

3 Large Class

Location: `Interface.java`, `InterfaceRenderer.java`

`Interface.java` handled many responsibilities in one class (state transitions, input, image loading, button animation/creation, and all screen rendering) and exceeded 700 lines. Rendering routines like `drawMainMenu`, `drawPaused`, `drawWin`, `drawGameOver`, `drawTutorial`, `drawScoreHeader`, `drawTimeHeader` were mixed with controller logic.

Why it is a problem

High class complexity and low cohesion made the class difficult to maintain and test. Adding or changing render behavior required touching UI control code, increasing regression risk.

Refactoring Performed

Identified all private rendering-only methods with no state side effects. Created `InterfaceRenderer` dedicated to all drawing logic. Moved all `draw*` methods and rendering constants there. Updated `paintComponent` to delegate render work through `InterfaceRenderer.render(...)` with only needed data. Removed now-redundant render dispatch methods from `Interface`. The six `testPaint*DoesNotThrow` tests in `InterfaceTest` all passed after the extraction, and `testPaintWinShowsPlaytimeAndScore` and `testPaintGameOverAfterPlaying` further confirmed that screen-specific data (balance, playtime) was still correctly passed through to the renderer.
Commit `b226cc3`: "refactor: extract renderer into InterfaceRenderer"

4 Switch Statement Overuse

Location: `Interface.java`

`Interface.java` used multiple `switch (state)` blocks over the same `GameState` enum (for button construction and drawing dispatch). Adding a new state required touching multiple switches, causing repetitive and error-prone updates.

Why it is a problem

This pattern violates Open/Closed-like flexibility and mixes control flow with UI layout data. It also made the code less extensible and harder to reason about because behaviour was split across duplicate switch statements.

Refactoring Performed

Identified button-layout switch block as primary duplication hotspot. Introduced `ButtonSpec` record to represent button metadata (position/size/image/action) independently from control flow. Replaced switch with `EnumMap<GameState, List<ButtonSpec>>` configured once in `configureButtonLayouts()`. State update now performs a single map lookup and loops through specs. The draw dispatch switch had already been removed in the renderer refactor. The button-count tests in `InterfaceTest` - `testMainMenuHasThreeButtons`, `testPlayingHasOneButton`, `testPausedHasFiveButtons`, `testWinHasThreeButtons`, `testGameOverHasT` and `testTutorialHasOneButton` - all passed, confirming every state still produces the correct button layout.
Commit `883b8e7`: "refactor: replace ui state switch with data driven button layout map"